

CSE-3212 — Numerical Analysis Lab

**Implementation and Comparative Analysis of the
Bisection, False Position, Newton–Raphson,
and Secant Methods for Finding Roots of
Nonlinear Equations**

Submitted By

Name : Farhan Bin Rabbani
Roll No. : 35
Session : 2022–2023

Submitted To

Palash Roy

Lecturer

Md. Aminul Kader Bulbul

Lecturer

Department of Computer Science and Engineering
University of Dhaka

1. Introduction

Root-finding is a fundamental problem in numerical analysis: given a continuous function $f(x)$, find a value x^* such that $f(x^*) = 0$. Analytical solutions exist only for a small class of equations; for most practical engineering and scientific problems one must resort to iterative numerical methods.

Four classical iterative methods are studied in this report:

- **Bisection Method** – a bracketing technique that repeatedly halves an interval $[x_l, x_u]$ in which a sign change of f is guaranteed. It is unconditionally convergent but converges slowly (linearly), roughly halving the error per iteration.
- **False Position (Regula Falsi) Method** – also a bracketing method. Instead of the midpoint it uses a linear interpolation between the two bracket endpoints, generally converging faster than bisection while retaining the sign-change guarantee.
- **Newton–Raphson Method** – an open method that uses both the function value and its derivative at the current estimate to project the next estimate. It exhibits quadratic convergence near a simple root, but requires a good initial guess and a non-zero derivative.
- **Secant Method** – an open method that approximates the derivative used in Newton–Raphson with a finite-difference ratio of two previous iterates, removing the need to compute $f'(x)$ analytically. It converges super-linearly (order ≈ 1.618).

2. Objectives

1. To understand the principles of the False Position, Newton–Raphson, and Secant methods.
2. To implement the iterative algorithms in a computer program.
3. To approximate the root of a nonlinear function to a desired accuracy.
4. To compare convergence behavior and limitations of these methods.

3. Algorithms

The test equation used across all four methods is:

$$f(x) = h^3 - 10h + 5e^{-h/2} - 2 = 0.$$

3.1. Bisection Method

1. Choose a bracket $[x_l, x_u]$ such that $f(x_l) \cdot f(x_u) < 0$.
2. Compute the midpoint $x_r = \frac{x_l + x_u}{2}$.
3. Evaluate $f(x_r)$. If $f(x_l) \cdot f(x_r) < 0$ set $x_u \leftarrow x_r$; otherwise set $x_l \leftarrow x_r$.
4. Compute the approximate relative error:

$$\varepsilon_a = \left| \frac{x_r^{\text{new}} - x_r^{\text{old}}}{x_r^{\text{new}}} \right| \times 100\%.$$

5. Repeat steps 2–4 until $\varepsilon_a \leq \varepsilon_s$.

3.2. False Position (Regula Falsi) Method

1. Choose a bracket $[x_l, x_u]$ such that $f(x_l) \cdot f(x_u) < 0$.
2. Compute the new estimate using linear interpolation:

$$x_r = \frac{x_l f(x_u) - x_u f(x_l)}{f(x_u) - f(x_l)}.$$

3. Update the bracket: if $f(x_l) \cdot f(x_r) < 0$ set $x_u \leftarrow x_r$; otherwise set $x_l \leftarrow x_r$.
4. Compute ε_a and repeat until $\varepsilon_a \leq \varepsilon_s$.

3.3. Newton–Raphson Method

1. Choose an initial guess x_0 .
2. Update:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}, \quad f'(x_i) \neq 0,$$

where $f'(x) = 3x^2 - 10 - 2.5e^{-x/2}$.

3. Compute $\varepsilon_a = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| \times 100\%$ and repeat until $\varepsilon_a \leq \varepsilon_s$.

3.4. Secant Method

1. Choose two initial estimates x_0 and x_1 .
2. Update using the secant formula:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

3. Compute $\varepsilon_a = \left| \frac{x_{k+1} - x_k}{x_{k+1}} \right| \times 100\%$ and repeat until $\varepsilon_a \leq \varepsilon_s$.

4. Implementation

All methods were implemented in Python 3 using `numpy`, `math`, and `matplotlib`. The stopping criterion for all methods is $\varepsilon_s = 0.001\%$.

4.1. Bisection Method

```

1 def f2(x):
2     return x**3 - 10*x + 5*m.exp(-x / 2) - 2
3
4 def bisection_problem2(xl, xu, es=0.001):
5     xr_old = 0
6     errors, iterations = [], []
7     i = 0
8
9     print(f"{'Iter':<5} {'x1':<10} {'xu':<10} {'xr':<10} {'f(xr)':<10}
10         {'ea (%)':<10}")
11     print("-" * 60)
12
13     while True:
14         xr = (xl + xu) / 2
15         fxr = f2(xr)
16         i += 1
17         ea = abs((xr - xr_old) / xr) * 100 if i > 1 else 100
18         errors.append(ea); iterations.append(i)
19         print(f"{'i':<5} {'x1':<10.5f} {'xu':<10.5f} {'xr':<10.5f}
20             {'fxr':<10.5f} {'ea':<10.5f}")
21
22         if ea <= es:
23             print(f"\nStopping criterion reached: ea = {ea:.4f}%")
24             break
25         if f2(xl) * f2(xr) < 0:
26             xu = xr
27         else:
28             xl = xr
29             xr_old = xr
30
31     return iterations, errors
32
33 iter2, err2 = bisection_problem2(0.1, 0.4)

```

Listing 1:]*BisectionMethod* – $-f(x)=x^3 - 10x + 5e^{-x/2} - 2$, bracket $[0.1, 0.4]$

4.2. False Position Method

```

1 def false(x):
2     return x**3 - 10*x + 5*m.exp(-x / 2) - 2
3
4
5 xl, xu, es, ea = 0.1, 0.4, 0.001, 100
6 old_root = xl; i = 0
7 iterations, approx_errors = [], []
8
9 if false(xl) * false(xu) > 0:
10     print("No root found!")
11 else:

```

```

12 print(f"{'Iter':<5} {'x1':<15} {'xu':<15} {'xr':<20} {'f(xr)':<20}
    {'ea':<15}")
13 while ea > es:
14     i += 1
15     root = (x1*false(xu) - xu*false(x1)) / (false(xu) - false(x1))
16     ea = abs((root - old_root) / root) * 100
17     print(f"{i:<5} {x1:<15.10f} {xu:<15.10f} {root:<20.10f}
        {false(root):<20.10f} {ea:<15.10f}")
18     iterations.append(i); approx_errors.append(ea)
19     if false(x1) * false(root) < 0:
20         xu = root
21     else:
22         x1 = root
23     old_root = root
24     print(f"\nThe root is {root:.10f} (Approx.)")

```

Listing 2:]FalsePositionMethod – $-bracket[0.1, 0.4]$

4.3. Newton–Raphson Method

```

1 def new(x):
2     return x**3 - 10*x + 5*m.exp(-x / 2) - 2
3
4 def new_der(x):
5     return 3*x**2 - 10 - 2.5*m.exp(-x / 2)
6
7 x0, es, ea, i = 1.5, 0.001, 100, 0
8 iterations, approx_errors = [], []
9
10 print(f"{'Iter':<5} {'xroot':<15} {'f(xroot)':<20} {'f\\'(xroot)':<20}
    {'ea':<15}")
11 while ea > es:
12     i += 1
13     if new_der(x0) == 0:
14         print("Derivative is 0"); break
15     xroot = x0 - new(x0) / new_der(x0)
16     ea = abs((xroot - x0) / xroot) * 100
17     print(f"{i:<5} {xroot:<15.10f} {new(xroot):<20.10f}
        {new_der(xroot):<20.10f} {ea:<15.10f}")
18     iterations.append(i); approx_errors.append(ea)
19     x0 = xroot
20

```

Listing 3: Newton–Raphson Method – initial guess $x_0 = 1.5$

4.4. Secant Method

```

1 def sec(x):
2     return x**3 - 10*x + 5*m.exp(-x / 2) - 2
3
4 x0, x1, es, ea, i = 1.5, 2.0, 0.001, 100, 0
5 iterations, approx_errors = [], []
6
7 print(f"{'Iter':<5} {'x0':<15} {'x1':<15} {'xk':<15} {'f(xk)':<20}
    {'ea':<15}")
8 while ea > es:
9

```

```

10     i += 1
11     xk = x1 - sec(x1) * ((x1 - x0) / (sec(x1) - sec(x0)))
12     ea = abs((xk - x1) / xk) * 100
13     print(f"{i:<5} {x0:<15.10f} {x1:<15.10f} {xk:<15.10f}
14           {sec(xk):<20.10f} {ea:<15.10f}")
15     iterations.append(i); approx_errors.append(ea)
16     x0, x1 = x1, xk

```

Listing 4: Secant Method – initial points $x_0 = 1.5$, $x_1 = 2.0$

5. Output and Analysis

All four methods were applied to the same nonlinear equation $f(x) = x^3 - 10x + 5e^{-x/2} - 2$ with stopping criterion $\varepsilon_s = 0.001\%$.

5.1. Bisection Method

Bracket: $[0.1, 0.4]$. The method required **17 iterations** to reach the stopping criterion ($\varepsilon_a = 0.0009\%$), converging to a root near $x^* \approx 0.2440$.

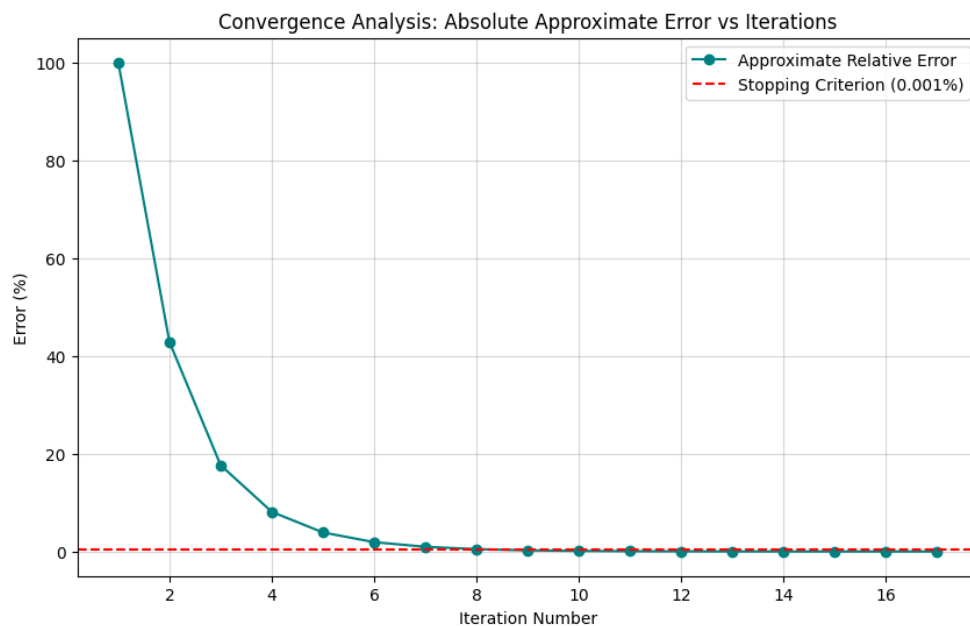


Figure 1: Bisection Method – Convergence of approximate relative error. The error decreases smoothly from 100% and crosses the 0.001% threshold at iteration 17, characteristic of linear (halving) convergence.

5.2. False Position Method

Bracket: $[0.1, 0.4]$. The method required **4 iterations** to converge, reaching $x^* \approx 0.2440223280$.

Table 1: False Position Method – iteration table

Iter	x_l	x_u	root	ε_a (%)
1	0.1000000000	0.4000000000	0.2464495373	59.4237
2	0.1000000000	0.2464495373	0.2440554796	0.9809
3	0.1000000000	0.2440554796	0.2440227739	0.0134
4	0.1000000000	0.2440227739	0.2440223280	0.0002

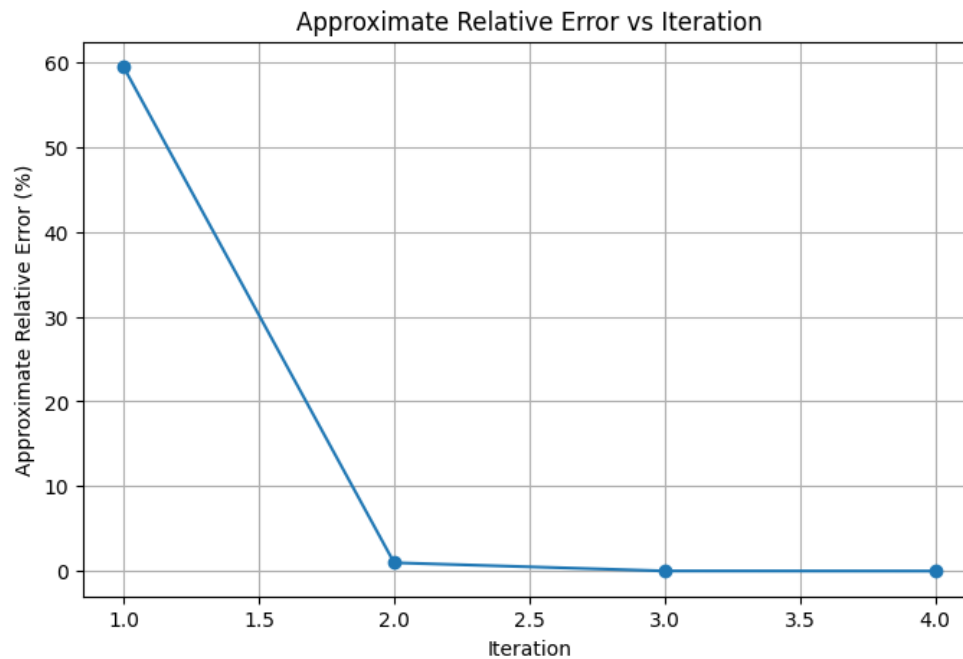


Figure 2: False Position Method – Approximate Relative Error vs. Iteration. The method converges rapidly in just 4 iterations for this well-behaved bracket.

5.3. Newton–Raphson Method

Initial guess: $x_0 = 1.5$. The method required **5 iterations** to converge, reaching $x^* \approx 0.2440223219$.

Table 2: Newton–Raphson Method – iteration table

Iter	x_{root}	$f(x_{\text{root}})$	ε_a (%)
1	-1.0419498509	15.7066	243.9609
2	0.3921591266	-1.7515	365.6957
3	0.2410790125	0.0354	62.6683
4	0.2440214011	0.0000	1.2058
5	0.2440223219	0.0000	0.0004

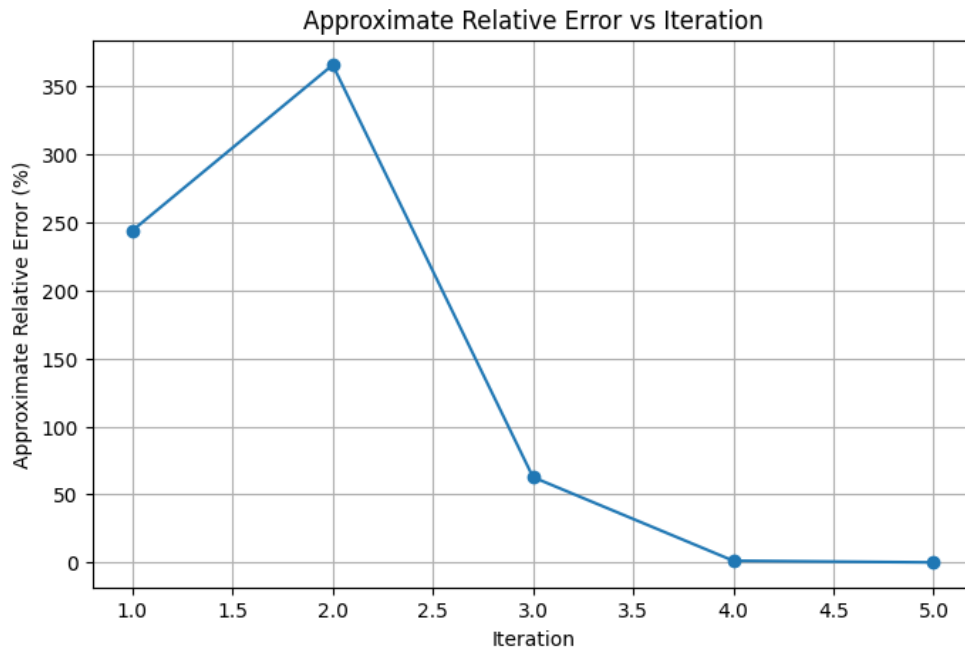


Figure 3: Newton–Raphson Method – Approximate Relative Error vs. Iteration. The initial guess $x_0 = 1.5$ is far from the target root, causing a large error spike at iteration 2 before quadratic convergence drives the error to near zero by iteration 5.

Note on convergence behaviour. The Newton–Raphson error initially *increases* (iteration 2: $\varepsilon_a \approx 365\%$) before collapsing rapidly. This is because the derivative $f'(1.5)$ is negative and the update step overshoots to $x \approx -1.04$, which is a poor intermediate estimate. From iteration 3 onward the iterate enters the convergence basin of the root and quadratic convergence takes over.

5.4. Secant Method

Initial points: $x_0 = 1.5$, $x_1 = 2.0$. The method required **8 iterations** to converge, reaching $x^* \approx -4.4744263967$.

Table 3: Secant Method – iteration table

Iter	x_0	x_1	x_k	ε_a (%)
1	1.5	2.0	−4.7752	141.88
2	2.0	−4.7752	−21.7934	78.09
3	−4.7752	−21.7934	−4.7758	356.33
4	−21.7934	−4.7758	−4.7763	0.01
5	−4.7758	−4.7763	−4.4964	6.22
6	−4.7763	−4.4964	−4.4762	0.45
7	−4.4964	−4.4762	−4.4744	0.04
8	−4.4762	−4.4744	−4.4744	0.0002

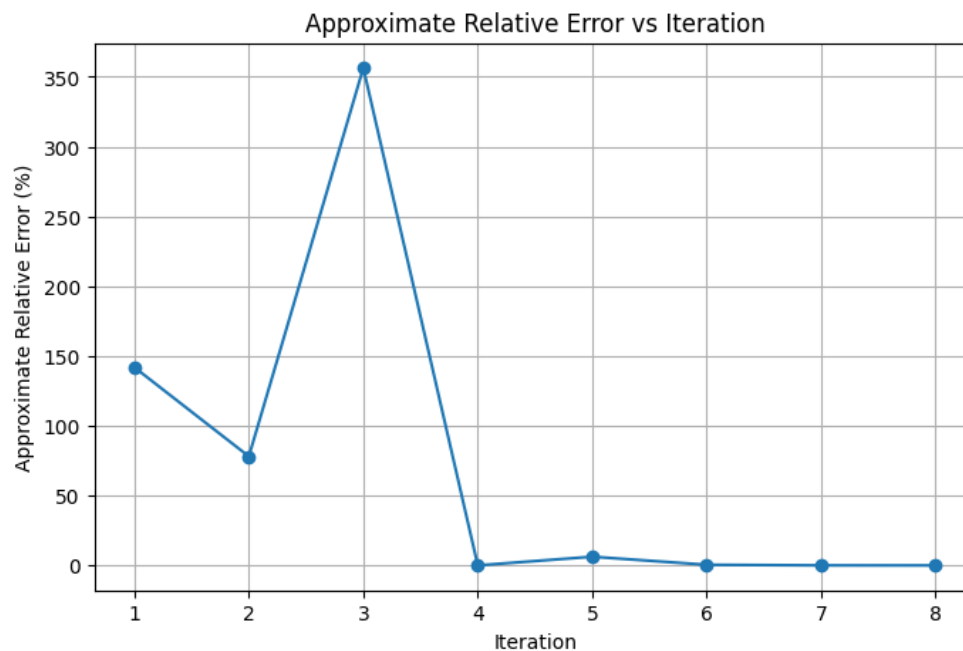


Figure 4: Secant Method – Approximate Relative Error vs. Iteration. Large oscillations in the early iterations reflect the fact that the two starting points $x_0 = 1.5$, $x_1 = 2.0$ are far from the converged root $x^* \approx -4.474$. The method stabilises and converges by iteration 8.

Note. The Secant method converged to a *different* root ($x^* \approx -4.474$) than the Bisection and False Position methods ($x^* \approx 0.244$). This is expected: the equation $f(x) = x^3 - 10x + 5e^{-x/2} - 2$ has multiple roots, and open methods converge to a root determined by the starting points rather than a bracketed interval. The Newton–Raphson starting from $x_0 = 1.5$ also passed through large negative values before settling back to the root near 0.244 – a consequence of the global nonlinearity of f .

5.5. Comparative Convergence Analysis

Figure 5 plots all four methods on a common axis. Because the error values span many orders of magnitude, the linear scale is used here to reflect the actual output from the programs; the large oscillations in Newton–Raphson and Secant are clearly visible alongside the smooth monotone decrease of the bracketing methods.

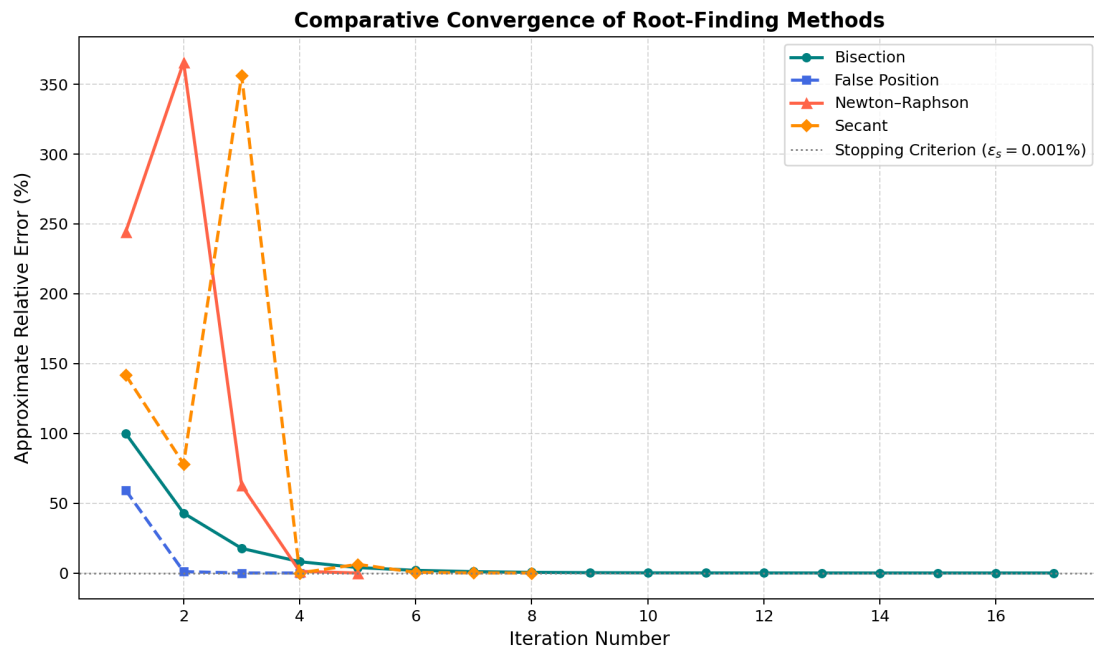


Figure 5: Comparative convergence of all four root-finding methods on a linear scale. The dotted horizontal line marks $\varepsilon_s = 0.001\%$. All methods use the same equation $f(x) = x^3 - 10x + 5e^{-x/2} - 2$ with the same stopping criterion.

Discussion. The following observations emerge from the comparison:

- **Bisection** requires the most iterations (17) due to its linear convergence. Every iteration reduces the bracket by exactly half, giving a monotonically decreasing, smooth error curve.
- **False Position** is the fastest bracketing method here, converging in only 4 iterations. The linear-interpolation estimate is generally much closer to the root than the midpoint.
- **Newton–Raphson** converges in 5 iterations but shows a dramatic initial spike (error $\approx 366\%$ at iteration 2) because the starting guess $x_0 = 1.5$ leads to an overshoot far from the root. Once in the convergence basin the quadratic rate quickly brings the error to zero.
- **Secant** requires 8 iterations due to large oscillations in the first three steps caused by the wide-spread initial points and the highly nonlinear shape of f in that region. Super-linear convergence then drives it to the stopping criterion.

Table 4: Summary of Method Characteristics and Observed Performance

Property	Bisection	False Pos.	Newton-R	Secant
Type	Bracketing	Bracketing	Open	Open
Convergence order	1 (linear)	≤ 1	2 (quadratic)	≈ 1.618
Derivative required	No	No	Yes	No
Two initial points	Yes	Yes	No	Yes
Guaranteed convergence	Yes	Yes	No	No
Iterations to converge	17	4	5	8
Converged root (x^*)	0.2440	0.2440	0.2440	-4.4744

6. Summary

This laboratory investigation implemented and analysed four iterative root-finding methods for the nonlinear equation $f(x) = x^3 - 10x + 5e^{-x/2} - 2$, with stopping criterion $\varepsilon_s = 0.001\%$. The key takeaways are:

1. **Bisection** is simple, robust, and guaranteed to converge, but its linear convergence rate required 17 iterations.
2. **False Position** was the fastest bracketing method, converging in just 4 iterations by using linear interpolation to produce a better estimate than the midpoint.
3. **Newton–Raphson** converged in 5 iterations after an initial overshoot. Its quadratic convergence rate makes it very efficient once the iterate enters the neighbourhood of the root, but it requires an analytical derivative and a sufficiently good initial guess.
4. **Secant** converged in 8 iterations to a different root of the equation than the bracketing methods, illustrating that open methods do not guarantee convergence to a specific root. Super-linear convergence made it competitive once the oscillations damped out.
5. In practice, method selection depends on the smoothness of f , the availability of f' , the quality of initial estimates, and whether convergence to a specific root (or at all) must be guaranteed.